i/p: [1, 2, 2, 3, 3]

o/p: [1, 2, 3, _, _, _], return 3.

i/p: [1, 2, 2, 3, 3]

o/p: [1, 2, 3]

TC = O(n)
AS = O(n)

[1, 2, 3]
0 1 2 3 4

[1, 2, 38, 3, 3]

[1, 24, 34, 58, 68, 5, 5, 6, 6]

[1, 2, 3, 5, 6, _ _ _ _ _]

TC: ____

[1, 1, 1, 2, 2, 3, 3, 4, 5, 5]

i: 0 ⟶ n-1
j: i+1 ⟶ n-1

Hashing.

Key → value.

Keys: 0 ⟶ n-1

0 → hello
1 → hi
10 → gfg
13 → geeks
⋮
200 → apple.
1000 → orange

1%37 = 1
1000%37 = 1

Collision

0	1		10		13
hello	hi	- - -	gfg	- -	geeks

0 → hello
1 → hi
2
5
⋮
13 → geeks
15 → apple.
36

Key → Hash Function → some valid index in the Hash Table.

% 37.

0 → hello
1 → hi → orange.
2
⋮
13
15
36

0 → (0, hello)
1 → (1, hi) → (1000, orange)
⋮
13 → (13, geeks),
15 →
36

Unordered Hash Maps.

	Avg	worst
Get op:	O(1)	O(n)
write op:	O(1)	O(1)
Delete op:	O(1)	O(n)

Ordered Hash Maps

	Avg	worst
Get	O(log n)	O(log n)
write	O(log n)	O(log n)
Delete	O(log n)	O(log n)

for ____ n^2
map. ____ n log n

[4, 2, -3, 1, 6]

i: 0 ⟶ n-1
sum = 0
j: i ⟶ n-1
sum += arr[j]
if sum == 0
print sum temp

BF
TC = O(n²)
AS = O(1)

arr → [4, 2, -3, 1, 6]
PS → 4 6 3 4 10

arr → [1, 2, -3, 6, 10]
PS → [1, 3, 0, 6, 10]

arr → [1, 2, -3]
PS → 1 3 0
SS → 0 -1 -3

[1, 2, -3, 6, 10]
0 1 3 0 6 10

sum = 0 & 3 & 0

[4, 2, -3, 1, 6]

sum = 0 & 0 & 4

TC → O(n)
AS → O(n)

Visited
0 ←
1
3

Visited
0
4 ←
6
3

[1, 0, 0, 1, 0, 1, 1]

arr → [1, -1, -1, 1, -1, 1, 1] → 0/1 = 8

PS: 0 1 0 -1 0 -1 0 1

4 → 2

[4, 2, -3, 1, 6, -3, -3]

PS → 4 6 3 4 10 7 4